

# CO5 – ASSESSMENT 2

## PSEUDOCODE WRITING TASK

NAME: A.ARCHISHMAN

REGNO: 192425341

COURSE CODE: ITA0502

COURSE NAME: COMPUTER VISION

### Q1. Real-Time Face Detection and Recognition

**Approach:** The following pseudocode describes a complete OpenCV-based real-time face detection and recognition pipeline. It supports multiple faces, minimizes unnecessary computation, and displays the recognized identity with a bounding box.

#### Detailed Pseudocode

```
BEGIN

LOAD OpenCV libraries
LOAD trained face detection model
LOAD trained face recognition / feature extraction model
LOAD registered face dataset and identity labels

INITIALIZE camera = VideoCapture(0)

IF camera is not opened THEN
    PRINT "Camera Error"
    STOP
END IF

WHILE TRUE

    frame = camera.read()

    IF frame is empty THEN
        BREAK
    END IF

    # Efficient preprocessing
    resized_frame = RESIZE(frame, required_width, required_height)
    gray_frame = CONVERT_TO_GRAYSCALE(resized_frame)
    gray_frame = APPLY_LIGHT_DENOISING(gray_frame)

    # Detect all faces
    face_boxes = DETECT_FACES(gray_frame)

    FOR each face_box IN face_boxes

        x, y, w, h = face_box
        face_crop = CROP(resized_frame, x, y, w, h)

        # Quality check
        IF face_crop is too small OR too blurred THEN
            CONTINUE
        END IF

        # Normalize and align
        face_crop = ALIGN_FACE(face_crop)
        face_crop = NORMALIZE_ILLUMINATION(face_crop)

        # Feature extraction
        feature_vector = EXTRACT_FEATURES(face_crop)

        # Recognition
        best_identity = "Unknown"
        best_score = INFINITY

        FOR each registered_person IN DATABASE
```

```

        score = CALCULATE_DISTANCE(
            feature_vector,
            registered_person.feature_vector)

        IF score < best_score THEN
            best_score = score
            best_identity = registered_person.name
        END IF
    END FOR

    IF best_score <= RECOGNITION_THRESHOLD THEN
        identity = best_identity
    ELSE
        identity = "Unknown"
    END IF

    # Visualization
    DRAW_RECTANGLE(resized_frame, face_box)
    DRAW_TEXT(resized_frame, identity, position_above_box)

END FOR

DISPLAY(resized_frame)

# Efficient loop control
key = WAIT_KEY(short_delay)

IF key == ESC OR key == 'q' THEN
    BREAK
END IF

END WHILE

RELEASE camera
CLOSE all windows

END

```

### Explanation of the Pseudocode

Explanation: 1. VideoCapture continuously acquires frames. 2. Resizing and light denoising reduce processing cost. 3. Face detection returns multiple face bounding boxes. 4. Each face is cropped, quality-checked, aligned and normalized. 5. Features are extracted and compared with registered identities. 6. A threshold prevents low-confidence matches from being forced to a person. 7. Bounding boxes and labels are displayed for every detected face. 8. A short wait and ESC/q termination provide controlled loop operation. 9. Processing a smaller frame and using efficient models helps maintain real-time performance.

## Q2. Vehicle Detection, Tracking and Counting

**Approach:** The following pseudocode detects vehicles inside a predefined ROI, tracks multiple vehicles, counts each vehicle exactly once when it crosses a virtual line, and displays IDs and the total count.

### Detailed Pseudocode

```
BEGIN

LOAD OpenCV libraries
LOAD vehicle detection model
INITIALIZE tracker
INITIALIZE camera = VideoCapture(0 or video_file)

SET ROI coordinates
SET virtual_counting_line
SET vehicle_count = 0
CREATE empty SET counted_track_ids

IF camera is not opened THEN
    PRINT "Video Error"
    STOP
END IF

WHILE TRUE

    frame = camera.read()

    IF frame is empty THEN
        BREAK
    END IF

    # Preprocessing
    resized_frame = RESIZE(frame, required_width, required_height)
    roi_frame = CROP(resized_frame, ROI)
    clean_frame = APPLY_LIGHT_DENOISING(roi_frame)

    # Vehicle detection
    detections = DETECT_VEHICLES(clean_frame)

    valid_detections = EMPTY LIST

    FOR each detection IN detections

        IF detection.confidence < DETECTION_THRESHOLD THEN
            CONTINUE
        END IF

        IF detection.class is CAR OR BUS OR TRUCK OR MOTORCYCLE THEN
            ADD detection TO valid_detections
        END IF

    END FOR

    # Multi-object tracking
    tracks = UPDATE_TRACKER(valid_detections)

    FOR each track IN tracks

        track_id = track.ID
        current_center = CALCULATE_CENTROID(track.bounding_box)
        previous_center = GET_PREVIOUS_CENTROID(track_id)

        # Restrict tracking to ROI
        IF current_center is outside ROI THEN
            CONTINUE
        END IF

        # Check virtual line crossing
        crossed = CHECK_LINE_CROSSING(
            previous_center,
            current_center,
            virtual_counting_line)
```

```

    IF crossed == TRUE THEN

        IF track_id NOT IN counted_track_ids THEN
            vehicle_count = vehicle_count + 1
            ADD track_id TO counted_track_ids
        END IF

    END IF

    # Visualization
    DRAW_RECTANGLE(frame, track.bounding_box)
    DRAW_TEXT(frame, "ID: " + track_id, track.bounding_box.top)
    DRAW_CIRCLE(frame, current_center)

END FOR

DRAW_LINE(frame, virtual_counting_line)
DRAW_TEXT(frame, "Vehicle Count: " + vehicle_count)

DISPLAY(frame)

# Efficient loop termination
key = WAIT_KEY(short_delay)

IF key == ESC OR key == 'q' THEN
    BREAK
END IF

END WHILE

RELEASE camera
CLOSE all windows

END

```

### Explanation of the Pseudocode

Explanation: 1. The ROI limits detection to the road area and reduces computation. 2. Confidence filtering removes weak vehicle detections. 3. The tracker assigns a persistent ID to each vehicle. 4. The centroid from the previous and current frame is used to detect line crossing. 5. The `counted_track_ids` set ensures that one vehicle is never counted twice. 6. Multiple vehicles can be processed in the same frame. 7. Resizing, ROI processing and a short loop delay help maintain near real-time performance. 8. ESC or 'q' terminates the loop safely and releases resources.